Ch12. 파드 스케쥴링

1. 노드 셀렉터

노드 셀렉터(nodeSelector)

- 가장 간단한 스케쥴링 옵션
- 파드의 .spec 필드에 설정
- **노드를 선택하는 기능**
- 파드가 클러스터 안 어떤 노드에서 실행될 지를 **키-값 쌍**으로 설정

kubectl get nodes --show-labels

- 클러스터 안 노드 레이블에 어떤 키-값 쌍이 있는지 확인

kubectl get nodes -L disktype

- disktype키에 대한 값을 확인

```
$ kubectl label nodes k8s-worker1 disktype=ssd
node/k8s-worker1 labeled
$ kubectl label nodes k8s-worker2 disktype=hdd
node/k8s-worker2 labeled
$ kubectl get nodes -L disktype
NAME          STATUS   ROLES           AGE    VERSION   DIS
KTYPE
k8s-master    Ready    control-plane   279d   v1.28.8
k8s-worker1   Ready    <none>          279d   v1.28.8   ssd
k8s-worker2   Ready    <none>          279d   v1.28.8   hdd
```

- 파드에 노드셀렉터를 설정하는 예(scheduling/nodeselector.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-nodeselector-pod
spec:
  containers:
  - name: kubernetes-nodeselector-pod
```

```
    image: arisu1000/simple-container-app:latest
    ports:
    - containerPort: 8080
  nodeSelector:
    disktype: ssd
```

- kubectl apply -f nodeselector.yaml
- kubectl get pods -o wide
  - nodeSelector 값에 의해 disktype=ssd인 worker1에서 파드가 실행

```
$ kubectl get pods -o wide
NAME                          READY   STATUS    RESTARTS
AGE   IP          NODE          NOMINATED NODE   READINESS
GATES
kubernetes-nodeselector-pod   1/1     Running   0
9s    10.46.0.1   k8s-worker1   <none>           <none>
```

---

2. 어피니티와 안티 어피니티

Affinity: 파드들을 함께 묶어서 같은 노드에서 실행하도록 설정

AntiAffinity: 파드들을 다른 노드에 나누어서 실행하도록 설정

2.1 노드 어피니티

Node affinity

- **노드 셀렉터와 비슷하게 노드의 레이블 기반**으로 파드를 스케쥴링한다.
- **노드 어피니티와 노드셀렉터를 함께 설정할 수도** 있으며, 이때는 노드 어피니티와 노드 셀렉터의 조건을 모두 만족하는 노드에 파드를 스케쥴링한다.
- 2가지 필드가 있다.
  - **requiredDuringSchedulingIgnoredDuringExecutio**n: 스케쥴링하는 동안 **꼭 필요한 조건**
  - **preferedDuringScheduingIgnoredDuringExecution**: 스케쥴링하는 동안 **만족하면 좋은 조건**. 꼭 이 조건을 만족해야 하는 것은 아니라는 뜻
  - 2 필드는 실행 중에 조건이 바뀌어도 무시 = 파드가 이미 스케쥴링되어 특정 노드에서 실행 중이라면 중간에 해당 노드의 조건이 변경되더라도 이미 실행 중인 파드는

그대로 실행된다는 뜻

- 노드 어피니티 설정 예(scheduling/node-affinity.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-nodeaffinity-pod
spec:
  containers:
  - name: kubernetes-nodeaffinity-pod
    image: arisu1000/simple-container-app:latest
    ports:
    - containerPort: 8080
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: beta.kubernetes.io/os
            operator: In
            values:
            - linux
            - window
          - key: disktype
            operator: Exists
          # - key: core
            # operator: Gt
            # values:
            # - "4"
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 10
        preference:
          matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
            - worker-node01
```

2.2 파드의 어피니티와 안티 어피니티

- 파드 사이의 어피니티와 안티 어피니티는 **디플로이먼트나 스테이트풀세트로 파드를 배포할 때 개별 파드 사이의 관계를 정의하는 용도**로 사용
- 안티 어피니티 설정 예(scheduling/pod-antiaffinity.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-cache
spec:
  selector:
    matchLabels:
      app: store
  replicas: 2
  template:
    metadata:
      labels:
        app: store
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - store
            topologyKey: "kubernetes.io/hostname"
      containers:
      - name: redis-server
        image: redis:3.2-alpine
```

- 파드 어피니티의 설정 예(scheduling/pod-affinity.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
```

```yaml
metadata:
  name: web-server
spec:
  selector:
    matchLabels:
      app: web-store
  replicas: 2
  template:
    metadata:
      labels:
        app: web-store
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - web-store
            topologyKey: "kubernetes.io/hostname"
        podAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - store
            topologyKey: "kubernetes.io/hostname"
      containers:
      - name: web-app
        image: nginx:1.12-alpine
```

3. 테인트와 톨러레이션 사용하기

쿠버네티스 클러스터의 특정 노드에 Taint를 설정할 수 있다.

- **Taint를 설정한 노드는 파드를 스케쥴링하지 않는다.**
- **Taint를 설정한 노드에 파드를 스케쥴링하려면 Toleration을 설정해야 한다.**
- 그럼 Taint는 Toleration에서 설정한 특정 파드들만 실행하고 다른 파드는 실행하지 못하게 한다.

Taint와 Toleration은 주로 **노드를 특정 역할만 하도록 만들 때 사용**한다.

kubectl taint nodes 노드명 키=값:효과

- Taint는 키, 값, 효과의 3가지로 구성된다.

```
$ kubectl taint nodes k8s-worker1 key01=value01:NoSchedule
\node/k8s-worker1 tainted
$ kubectl describe nodes k8s-worker1 | grep -i taint
Taints:             key01=value01:NoSchedule
```

- **kubectl describe nodes 노드명**
  - Taint 설정 여부를 확인
- 테인트 설정을 확인하는 디플로이먼트 설정 예(deployment/deployment-sample.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-simple-app
  labels:
    app: kubernetes-simple-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kubernetes-simple-app
  template:
    metadata:
      labels:
        app: kubernetes-simple-app
    spec:
```

```
      containers:
      - name: kubernetes-simple-app
        image: arisu1000/simple-container-app:latest
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: 50m
```

- kubectl apply -f deployment-sample.yaml
- kubectl get pods -o wide
  - **Taint가 설정된 노드에는 파드가 실행되지 않는다.**
  - **Taint가 설정된 노드에 파드를 실행시키려면 Toleration을 설정해야 한다.**
  - 다음은 .tolerations 필드를 추가로 설정한 예이다.
- 톨러레이션 설정을 확인하는 디플로이먼트 설정 예(deployment/deployment-toleration.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-simple-app
  labels:
    app: kubernetes-simple-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: kubernetes-simple-app
  template:
    metadata:
      labels:
        app: kubernetes-simple-app
    spec:
      containers:
      - name: kubernetes-simple-app
        image: arisu1000/simple-container-app:latest
```

```
        ports:
        - containerPort: 8080
      tolerations:
      - key: "key01"
        operator: "Equal"
        value: "value01"
        effect: "NoSchedule"
```

- kubectl apply -f deployment-toleration.yaml
- kubectl get pods -o wide
  - 테인트가 설정된 노드에서도 파드가 실행됨을 확인

```
$ kubectl get po -o wide
NAME                                        READY   STATUS
RESTARTS   AGE   IP          NODE          NOMINATED NODE
READINESS GATES
kubernetes-simple-app-6bd6d87df8-jgvd7   1/1     Running
0          6s    10.32.0.4   k8s-worker2   <none>
<none>
kubernetes-simple-app-6bd6d87df8-t9zpf   1/1     Running
0          9s    10.46.0.1   k8s-worker1   <none>
<none>
```

kubectl taint nodes 노드명 키:효과-

- Taint 설정을 삭제

```
$ kubectl taint nodes k8s-worker1 key01:NoSchedule-
node/k8s-worker1 untainted
```

3.1 테인트와 톨러레이션의 하위 필드

- 테인트에 사용하는 .key 필드 값의 첫문자는 영문이나 숫자로 시작해야 하고 영문, 숫자, 하이픈(-), 점(.), 밑줄(_) 등을 사용할 수 있다. 253자까지 작성할 수 있다.
- .value 필드 값은 키와 마찬가지로 조건으로 작성할 수 있지만 63자까지만 작성할 수 있다는 차이가 있다.
- .effect 필드 값은 NoSchedule, PreferNoSchedule, NoExecute로 설정

- **NoSchedule**
  - 톨러레이션 설정이 없으면 파드를 스케쥴링하지 않는다.
  - **기존에 실행되던 파드에는 적용되지 않는다.**
- **PreferNoSchedule**
  - 톨러레이션 설정이 없으면 파드를 스케쥴링하지 않는다.
  - 하지만 클러스터 안 **자원이 부족하면 테인트를 설정한 노드에서도 파드를 스케쥴링**할 수 있다.
- **NoExecute**
  - 톨러레이션 설정이 없으면 새로운 파드를 스케쥴링하지 않으며, **기존 파드도 테인트 설정을 무시할 수 있는 톨러레이션 설정이 없으면 종료시킨다.**

- .toleration 하위의 .key, .value, .effect 필드 값으로는 원하는 테인트의 설정값을 넣는다.
- .operator 필드 값은 **Equal**과 **Exists**가 있다.

---

4. 클러스터를 관리하는 커든과 드레인

cordon, drain

- 특정 노드에 있는 **파드들을 모두 다른 노드로 옮기거나 특정 노드에 파드들을 스케쥴링하지 않도록 제한**할 필요가 있을 때 사용하는 명령어

4.1 커든 설정하기

kubectl cordon 노드명

- 지정된 노드에 추가로 파드를 스케쥴링해서 실행하지 않도록 한다.

```
$ kubectl get nodes
NAME          STATUS   ROLES           AGE    VERSION
k8s-master    Ready    control-plane   281d   v1.28.8
k8s-worker1   Ready    <none>          281d   v1.28.8
k8s-worker2   Ready    <none>          281d   v1.28.8
$ kubectl cordon k8s-worker1
node/k8s-worker1 cordoned
$ kubectl get nodes
NAME          STATUS                      ROLES           AG
```

```
E    VERSION
k8s-master    Ready                      control-plane   28
1d   v1.28.8
k8s-worker1   Ready,SchedulingDisabled   <none>          28
1d   v1.28.8
k8s-worker2   Ready                      <none>          28
1d   v1.28.8
```

- kubectl get nodes 명령으로 이름 확인 및 상태 확인
  - STATUS 항목에 파드 스케쥴링을 준비 중인 Ready와 함께 이 노드에 더 이상 파드를 스케쥴링하지 않는다는 SchedulingDisabled가 추가된 것을 확인
- kubectl create deploy deploy-test --image=nginx:1.14 --replicas=2

```
$ kubectl get deploy,po,rs
NAME                          READY   UP-TO-DATE   AVAILABL
E   AGE
deployment.apps/deploy-test   2/2     2            2
113s

NAME                                 READY   STATUS    RESTAR
TS   AGE
pod/deploy-test-5cd79d8497-nfmgm   1/1     Running   0
71s
pod/deploy-test-5cd79d8497-zh5bm   1/1     Running   0
71s

NAME                                       DESIRED   CURRENT
READY   AGE
replicaset.apps/deploy-test-5cd79d8497   2         2
2       113s
guru@k8s-master:~$ kubectl get pods -o wide
NAME                           READY   STATUS    RESTARTS
AGE   IP          NODE          NOMINATED NODE   READINESS
GATES
deploy-test-5cd79d8497-nfmgm   1/1     Running   0
84s   10.32.0.2   k8s-worker2   <none>           <none>
```

```
deploy-test-5cd79d8497-zh5bm   1/1     Running   0
84s   10.32.0.4   k8s-worker2   <none>           <none>
```

kubectl uncordon 노드명

- 노드에 정상적으로 파드를 스케쥴링하는 명령

```
$ kubectl uncordon k8s-worker1
node/k8s-worker1 uncordoned
$ kubectl get nodes
NAME          STATUS   ROLES           AGE    VERSION
k8s-master    Ready    control-plane   281d   v1.28.8
k8s-worker1   Ready    <none>          281d   v1.28.8
k8s-worker2   Ready    <none>          281d   v1.28.8
```

4.2 드레인 설정하기

kubectl drain 노드명

- 노드 관리 등의 이유로 **지정된 노드에 있는 파드들을 다른 노드로 이동**시키는 명령
- **먼저 새로운 파드를 노드에 스케쥴링해서 실행하지 않도록** 한다.
- 기존 해당 노드에서 실행 중이던 파드들을 삭제한다.
- **--ignore-daemonsets=true**
  - 데몬세트로 실행한 파드들을 무시하고 드레인 설정을 적용
- **--force**
  - 드레인 설정을 적용해 파드를 강제로 삭제

```
$ kubectl drain k8s-worker2 --ignore-daemonsets=true --forc
e
node/k8s-worker2 cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/kube-
proxy-7rl68, kube-system/weave-net-sbnc5
evicting pod default/deploy-test-5cd79d8497-zh5bm
evicting pod default/deploy-test-5cd79d8497-nfmgm
pod/deploy-test-5cd79d8497-nfmgm evicted
pod/deploy-test-5cd79d8497-zh5bm evicted
node/k8s-worker2 drained
```

```
$ kubectl get nodes
NAME          STATUS                     ROLES           AG
E    VERSION
k8s-master    Ready                      control-plane   28
1d   v1.28.8
k8s-worker1   Ready                      <none>          28
1d   v1.28.8
k8s-worker2   Ready,SchedulingDisabled   <none>          28
1d   v1.28.8
```

- kubectl get nodes
  - 상태 확인
- kubectl get pods -A
  - 클러스터 안 자원 상태 확인

kubectl uncordon 노드명

- 드레인 설정을 해제

```
$ kubectl uncordon k8s-worker2
node/k8s-worker2 uncordoned
$ kubectl get nodes
NAME          STATUS   ROLES           AGE    VERSION
k8s-master    Ready    control-plane   281d   v1.28.8
k8s-worker1   Ready    <none>          281d   v1.28.8
k8s-worker2   Ready    <none>          281d   v1.28.8
```